

ZASTOSOWANIE REKURENCYJNYCH SIECI NEURONOWYCH DO PROGNOZOWANIA ZATŁOCZENIA W RUCHU MIEJSKIM (SMART CITY)

Gevorgyan Alis
Robaczewski Damian
Truszkowski Mateusz

Uniwersytet Kazimierza Wielkiego
Wydział Informatyki
ul. M. Kopernika 1, 85-064 Bydgoszcz
e-mail:

alis.gevorgyan@student.ukw.edu.pl
damian.robaczewski@student.ukw.edu.pl
mateusz.truszkowski@student.ukw.edu.pl

Streszczenie: Projekt dotyczy problemu prognozowania zatłoczenia w ruchu miejskim w kontekście smart city, a jego celem było zaprojektowanie i implementacja prototypowego systemu opartego na rekurencyjnych sieciach neuronowych do przewidywania natężenia w ruchu w mieście Londyn. Realizacja obejmowała analizę wymagań, pozyskanie i wstępne przetwarzanie publicznych danych, projekt architektury RNN, implementację i walidację modelu oraz integrację z modulem wizualizacji ruchu. Zastosowano metody uczenia głębokiego do modelowania danych czasowych oraz narzędzia analityczne i wizualizacyjne. Projekt doprowadził do stworzenia spójnego rozwiązania demonstracyjnego. Wnioski wskazują na zasadność wykorzystania sieci rekurencyjnych w prognozowaniu ruchu miejskiego oraz ich potencjał w systemach inteligentnego transportu.

Słowa kluczowe: prognozowanie ruchu miejskiego; smart city; rekurencyjne sieci neuronowe; uczenie maszynowe; dane czasowe; inteligentne systemy transportowe;

0. WSTĘP

Współczesne miasta mierzą się z narastającym problemem zatłoczenia ruchu drogowego, który wpływa negatywnie na jakość życia mieszkańców, środowisko oraz efektywność transportu. W ramach koncepcji smart city coraz większe znaczenie zyskują narzędzia analityczne umożliwiające prognozowanie natężenia ruchu i wspierające zarządzanie infrastrukturą miejską. Rekurencyjne sieci neuronowe są często stosowane do analizy danych czasowych ruchu drogowego [1-2]. Jednocześnie wskazuje się na istniejące różnice w podejściach do projektowania modeli oraz ograniczenia wynikające z jakości i dostępności danych.

Celem niniejszej pracy jest zaprojektowanie i implementacja modelu opartego na rekurencyjnych sieciach neuronowych do prognozowania zatłoczenia w ruchu miejskim na przykładzie Londynu. Postawiono pytanie badawcze, czy

zastosowanie RNN pozwala na skuteczne odwzorowanie zależności czasowych w danych oraz może stanowić wsparcie dla systemów inteligentnego transportu?

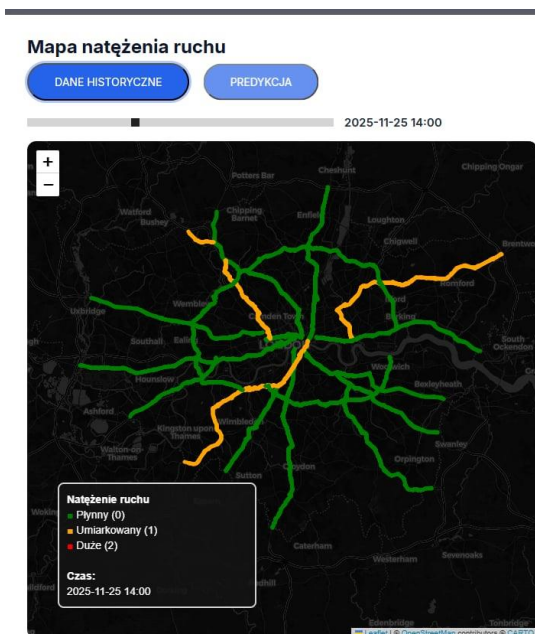
1. OGRANICZENIA

Ograniczenia zastosowanego rozwiązania wynikają przede wszystkim z dostępnych zasobów sprzętowych oraz przyjętych założeń projektowych. Ze względu na ograniczoną moc obliczeniową środowiska, proces trenowania oraz generowania predykcji został przeprowadzony na wybranym przedziale czasowym, co pozwoliło na zachowanie rozsądnego czasu obliczeń. Zastosowane ograniczenia mają charakter techniczny i konfiguracyjny, a nie architektoniczny. Wymagałoby to niewielkich modyfikacji konfiguracyjnych, aby rozszerzyć działanie systemu na dłuższy horyzont czasowy lub uruchomić model w trybie bieżącym.

2. MATERIAŁY I METODY

W ramach realizacji projektu wykorzystano publicznie dostępne dane dotyczące natężenia ruchu miejskiego na obszarze miasta Londyn, pochodzące z otwartych źródeł administracji publicznej. Dane obejmowały pomiary ruchu drogowego, informacje czasowe i dane lokalizacyjne, co umożliwiło analizę zależności sekwencyjnych i czasowych. Zbiory danych zostały pobrane, a następnie poddane wstępnemu przetwarzaniu obejmującemu, czyszczenie danych, normalizację oraz transformację do postaci sekwencji czasowych odpowiednich dla modeli rekurencyjnych.

Do implementacji wykorzystano język Python oraz biblioteki PyTorch do budowy modeli rekurencyjnych sieci neuronowych, pandas, NumPy do analizy i przetwarzania danych, scikit-learn do podziału danych i walidacji modelu, a także matplotlib do wizualizacji wyników, HTML z CSS do stworzenia strony internetowej prezentującej założenia, opis i uzyskane rezultaty projektu, a także Git/GitHub do kontroli wersji i współpracy zespołowej. Model RNN został zaprojektowany w oparciu o charakter danych czasowych, a jego parametry były dostrajane na podstawie wyników walidacji, z uwzględnieniem ryzyka nadmiernego dopasowania. Skuteczność modelu oceniono na wydzielonym zbiorze testowym.



Rysunek 1. Przykładowa wizualizacja danych ruchu drogowego, źródło: opracowanie własne.

Rysunek przedstawia przykładową wizualizację ruchu drogowego w funkcji czasu wraz z prognozą wygenerowaną przez model oparty na rekurencyjnej sieci neuronowej. Na wykresie zaprezentowano rzeczywiste wartości natężenia ruchu oraz odpowiadające im wartości prognozowane, co umożliwia jakościową ocenę dopasowania modelu do danych historycznych. wizualizacja pozwala na identyfikację trendów oraz cyklicznych zmian ruchu miejskiego, a także ilustruje zdolność modelu do odwzorowywania zależności czasowych.

3. OPIS IMPLEMENTACJI

3.1. Pozyskiwanie danych z API (*collector.py*)

Moduł *collector.py* odpowiada za cykliczne pozyskiwanie danych historycznych dotyczących natężenia ruchu miejskiego z publicznie dostępnego interfejsu API Transport for London (TfL). Dane pobierane są w określonym interwale czasowym (co jedną minutę), a następnie zapisywane w postaci pliku CSV, który stanowi zbiór danych wejściowych dla dalszych etapów przetwarzania i analizy. Każda porcja pobranych danych zostaje opatrzona znacznikiem czasu w standardzie UTC, co umożliwia zachowanie chronologii obserwacji niezbędnej do dalszego przetwarzania danych sekwencyjnych. Moduł uwzględnia podstawową obsługę błędów, co pozwala na stabilne działanie procesu zbierania danych.

```
df = get_tfl_roads()
if df is not None and not df.empty:
    df["timestamp"] = datetime.utcnow()
    file_exists = OUTPUT_FILE.exists()
    df.to_csv(str(OUTPUT_FILE),
              mode="a", header=not file_exists,
              index=False)

print(f"{datetime.now().strftime('%H:%M:%S')} - Zapisano {len(df)} rekordów")
```

Fragment kodu realizujący cykliczne pobieranie danych z interfejsu API

3.2. Wstępne przetworzenie danych (*preprocess.py*)

Moduł *preprocess.py* odpowiada za przygotowanie surowych danych dotyczących natężenia ruchu miejskiego do postaci umożliwiającej dalsze przetwarzanie oraz uczenie modelu rekurencyjnego. Dane wejściowe wczytywane są z

pliku CSV wygenerowanego na etapie pozyskiwania danych i poddawane kolejnym operacjom czyszczenia oraz transformacji. W pierwszym etapie realizowane jest mapowanie opisowych statusów ruchu drogowego [„Good”, „Serious”, „Severe”] na wartości liczbowe [0, 1, 2]. Pozwala to na przekształcenie danych jakościowych do postaci numerycznej, wymaganej przez algorytmy uczenia maszynowego. Wartości nieznane lub niesklasyfikowane zapisywane są do osobnego pliku, co umożliwia ich późniejszą analizę bez utraty informacji. Następnie dane pozbawione poprawnych wartości są usuwane z dalszego przetwarzania. Kolejnym krokiem jest konwersja znaczników czasu do formatu datetime oraz sortowanie danych według identyfikatora drogi i czasu. Zapewnia to zachowanie poprawnej chronologii obserwacji, co jest kluczowe w przypadku danych sekwencyjnych. Na tej podstawie generowane są cechy czasowe opisujące porę dnia oraz dzień tygodnia, które następnie kodowane są w sposób cykliczny z wykorzystaniem funkcji sinus i cosinus. Podejście to umożliwia zachowanie ciągłości zmiennych czasowych i lepsze odwzorowanie ich charakteru w modelu rekurencyjnym. Na końcu moduł zapisuje przetworzone dane do pliku wynikowego, który stanowi bezpośrednie wejście do etapu tworzenia sekwencji czasowych oraz trenowania modelu predykcyjnego.

```
df["hour"] = df["timestamp"].dt.hour
df["dow"] = df["timestamp"].dt.dayofweek
df["hour_sin"] = np.sin(2 * np.pi *
df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi *
df["hour"] / 24)

df["dow_sin"] = np.sin(2 * np.pi *
df["dow"] / 7)
df["dow_cos"] = np.cos(2 * np.pi *
df["dow"] / 7)
```

Fragment kodu generujący cykliczne cechy czasowe (pora dnia i dzień tygodnia) z wykorzystaniem funkcji trygonometrycznych.

3.3. Tworzenie sekwencji czasowych (make_sequences.py)

Moduł make_sequences.py odpowiada za przekształcanie przetworzonych danych czasowych do postaci sekwencji wejściowych i wyjściowych, wymaganych przez rekurencyjną sieć neuronową. Jego głównym celem jest przygotowanie danych w taki sposób, aby model mógł uczyć się zależności czasowych występujących w natężeniu ruchu

miejskiego. Zdefiniowane zostały parametry długości sekwencji wejściowej oraz wyjściowej. Sekwencja wejściowa obejmuje 72 kolejne obserwacje czasowe, natomiast sekwencja wyjściowa zawiera 48 przyszłych wartości natężenia ruchu, które model ma prognozować. Takie podejście umożliwia realizację krótkoterminowej predykcji wielokrokowej. Dodatkowo określono zestaw cech wejściowych, obejmujący zarówno natężenie ruchu, jak i zakodowane cyklicznie cechy czasowe. Dane zostały podzielone na trzy niezależne zbiory: treningowy, testowy oraz prognostyczny. Podział czasowy zapobiega przenikaniu informacji z przyszłości do procesu uczenia modelu i pozwala na rzetelną ocenę jakości predykcji. Sekwencje budowane są osobno dla każdej drogi, co umożliwia zachowanie lokalnych zależności czasowych charakterystycznych dla poszczególnych odcinków sieci drogowej. Proces tworzenia sekwencji polega na przesuwaniu okna czasowego po danych i generowaniu par wejście-wyjście. Dla każdej sekwencji wejściowej tworzony jest odpowiadający jej wektor wartości docelowych. Gotowe sekwencje są następnie konwertowane do tensorów biblioteki PyTorch i zapisywane do plików binarnych, które stanowią bezpośrednie dane wejściowe dla etapu trenowania i ewaluacji modelu.

```
for i in range(len(df_road) - INPUT_LEN
- OUTPUT_LEN + 1):
    X.append(values[i : i + INPUT_LEN])
    y.append(target[i + INPUT_LEN : i +
INPUT_LEN + OUTPUT_LEN])
```

Mechanizm budowania sekwencji wejściowych i wyjściowych z wykorzystaniem przesuwne okna czasowego.

Na potrzeby realizacji projektu oraz ze względu na ograniczenia sprzętowe, zakres danych wykorzystywanych w procesie trenowania, testowania oraz generowania prognoz został ograniczony do wybranego przedziału czasowego. Podział ten obejmował dane treningowe z okresu 9.11.2025 - 4.12.2025, dane testowe - 5.12.2025 - 11.12.2025, oraz zakres prognozy - 12.12.2025 - 15.12.2025. Takie rozwiązanie pozwoliło na przeprowadzenie pełnego procesu uczenia i ewaluacji modelu w dostępnych warunkach obliczeniowych. Wystarczy drobna modyfikacja w kodzie, aby umożliwić modelowi działanie w trybie bieżącym.

```
TRAIN_START = "2025-11-09"
TRAIN_END = "2025-12-04"
TEST_START = "2025-12-05"
```

```
TEST_END = "2025-12-11"
FORECAST_START = "2025-12-12"
FORECAST_END = "2025-12-15"
```

3.4. Implementacja modelu predykcyjnego (model.py)

Moduł model.py zawiera definicję rekurencyjnej sieci neuronowej wykorzystywanej do prognozowania natężenia ruchu miejskiego. Model został zaimplementowany z użyciem biblioteki PyTorch i opiera się na architekturze LSTM (Long Short-Term Memory), która jest szczególnie dobrze przystosowana do analizy danych sekwencyjnych oraz uchwycenia długoterminowych zależności czasowych.

Definicja warstwy LSTM:

```
self.lstm = nn.LSTM(
    input_size=input_size,
    hidden_size=hidden_size,
    num_layers=num_layers,
    batch_first=True,
    dropout=0.2
)
```

input_size - określa liczbę cech wejściowych w jednym kroku czasowym,

hidden_size - liczba neuronów w stanie ukrytym LSTM,

num_layers - zastosowanie dwóch warstw LSTM umożliwia modelowi uczenie się bardziej złożonych reprezentacji czasowych niż w przypadku pojedynczej warstwy

batch_first=True - oznacza, że dane wejściowe mają postać (batch_size, sequence_length,

input_size), co jest spójne z przygotowanymi wcześniej sekwencjami danych,

dropout=0.2 - mechanizm dropout losowo dezaktywuje 20% połączeń między warstwami LSTM w trakcie uczenia, co ogranicza ryzyko nadmiernego dopasowania (overfitting).

Warstwy w pełni połączone:

```
self.fc = nn.Sequential(
    nn.Linear(hidden_size, 64),
    nn.ReLU(),
    nn.Linear(64, 48)
)
```

nn.Linear(hidden_size, 64) - redukuje liczbę neuronów z 128 do 64, co stabilizuje proces uczenia,

nn.ReLU() - funkcja aktywacji wprowadza nieliniowość do modelu, umożliwiając lepsze odwzorowania złożonych zależności w danych,

nn.Linear(64,48) - warstwa wyjściowa generuje prognozę 48 kolejnych kroków czasowych, co odpowiada założonej długości predykcji

Dobór liczby warstw oraz liczby neuronów w stanie ukrytym stanowi kompromis pomiędzy

zdolnością modelu do uczenia złożonych zależności czasowych a ograniczeniami sprzętowymi środowiska projektowego.

```
def forward(self, x):
    out, _ = self.lstm(x)
    last = out[:, -1, :]
    return self.fc(last)
```

W procesie propagacji w przód (forward pass) sekwencje wejściowe przekazywane są do warstwy LSTM, która generuje sekwencję stanów ukrytych. Następnie wykorzystywany jest ostatni stan czasowy, zawierający skondensowaną informację o całej sekwencji wejściowej.

Stan ten przekazywany jest do warstw w pełni połączonych, które generują ostateczną prognozę natężenia ruchu.

3.5. Trenowanie modelu predykcyjnego (train.py)

Moduł train.py odpowiada za proces trenowania rekurencyjnej sieci neuronowej na wcześniej przygotowanych sekwencjach danych czasowych. W pierwszym etapie wczytywane są sekwencje treningowe oraz walidacyjne, które zostały wygenerowane w module tworzenia sekwencji. Dane te są następnie opakowywane w obiekty DataLoader, co umożliwia ich efektywne przetwarzanie w postaci mini-batchy. Model inicjalizowany jest na podstawie wcześniej zdefiniowanej architektury LSTM, przy czym liczba cech wejściowych określana jest dynamicznie na podstawie danych treningowych. Do optymalizacji wag sieci wykorzystano algorytm Adam, który dobrze sprawdza się w zadaniach regresji z wykorzystaniem sieci neuronowych. Jako funkcję straty zastosowano Mean Square Error (MSE), co jest standardowym wyborem w przypadku prognozowania wartości ciągłych. Proces trenowania jest realizowany iteracyjnie w pętli epok. W każdej epoce model uczony jest na zbiorze treningowym, po czym jego jakość oceniana jest na zbiorze walidacyjnym zachowującym porządek czasowy danych. W celu ograniczenia ryzyka nadmiernego dopasowania zastosowano mechanizm early stopping, który przerywa trening, gdy jakość modelu na zbiorze walidacyjnym przestaje się poprawiać przez określoną liczbę epok. Najlepszy model zapisywany jest do pliku i wykorzystywany w dalszym etapie ewaluacji.

```

if val_loss < best_val:
    best_val = val_loss
    patience = 0
    torch.save(model.state_dict(),
model_path)
else:
    patience += 1
    if patience >= PATIENCE:
        print("Early stopping")
        break

```

Fragment kodu przedstawiający mechanizm wczesnego zatrzymywania treningu (early stopping) zapobiegający nadmiernemu dopasowaniu modelu.

3.6. Ewaluacja modelu predykcyjnego (evaluate.py)

Moduł evaluate.py odpowiada za ocenę jakości działania wytrenowanego modelu predykcyjnego na zbiorze testowym. W pierwszym etapie wczytywane są wcześniej przygotowane sekwencje testowe oraz zapisany model sieci neuronowej. Następnie model uruchamiany jest w trybie ewaluacyjnym, a predykcje generowane są partiami z wykorzystaniem mechanizmu DataLoader, co pozwala na efektywne przetwarzanie danych oraz ograniczenie zużycia pamięci. Jakość modelu oceniana jest dwuetapowo. Liczone są metryki regresyjne: MSE, RMSE, MAE, które pozwalają na ilościową ocenę dokładności prognozowanych wartości ciągłych natężenia ruchu. Następnie, w celu ułatwienia interpretacji wyników, wartości ciągłe zostają zdyskretyzowane do klas odpowiadających poziomom natężenia ruchu (good, serious, severe). Na tej podstawie wyznaczana jest macierz pomyłek oraz raport klasyfikacji, umożliwiające analizę skuteczności modelu z punktu widzenia klasyfikacyjnego. Takie podejście pozwala na kompleksową ocenę modelu zarówno w ujęciu numerycznym, jak i interpretacyjnym.

```

mse = mean_squared_error(y_true_flat,
y_pred_flat)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_true_flat,
y_pred_flat)

```

Fragment kodu przedstawiający metryki regresyjne

3.7. Generowanie prognoz natężenia ruchu (predict.py)

Moduł predict.py odpowiada za wygenerowanie końcowych prognoz natężenia ruchu miejskiego z wykorzystaniem wytrenowanego modelu rekurencyjnej sieci neuronowej. W pierwszym etapie wczytywany jest zapisany model predykcyjny oraz przetworzone dane wejściowe zawierające informacje o ruchu drogowym i cechy czasowe. Dane są następnie sortowane chronologicznie oraz dzielone według poszczególnych odcinków dróg. Dla każdej drogi wybierane jest ostatnie okno czasowe obejmujące 72 godziny historycznych obserwacji, które stanowi wejście do modelu LSTM. Na tej podstawie generowana jest prognoza wielokrokowa obejmująca 48 następnych godzin. Predykcja realizowana jest w postaci wartości ciągłych, które następnie ograniczane są do przyjętego zakresu natężenia ruchu oraz mapowane na klasy opisujące poziom ruchu.

```

with torch.no_grad():
    pred_cont=model(X).cpu().numpy().flatten()

```

Generowanie prognozy wielokrokowej z wykorzystaniem wytrenowanego modelu LSTM

4. WYNIKI

Jakość modelu oceniono na podstawie macierzy pomyłek oraz raportu klasyfikacji. Uzyskane wyniki wskazują na bardzo wysoką skuteczność modelu w rozróżnianiu klas good oraz serious, przy jednoczesnym braku próbek klasy severe w zbiorze testowym. Dokładność ogólna modelu wyniosła 99%, co potwierdza wysoką skuteczność predykcyjną w analizowanym problemie.

- Klasa good została sklasyfikowana z bardzo wysoką precyzją (0,99) i niemal idealnym recall (1,00), co oznacza poprawne rozpoznawanie zdecydowanej większości obserwacji;
- Klasa serious osiągnęła precyzję na poziomie 0,91 oraz recall równy 0,84, co wskazuje na dobrą, choć mniej doskonałą skuteczność w porównaniu do klasy dominującej;
- Klasa severe nie występowała w danych testowych, w związku z czym nie było możliwe obliczenie miarodajnych metryk dla tej kategorii;

Niska wartość średnich makro (macro avg) wynika z braku obserwacji jednej z klas, natomiast średnie ważone (weighted avg) odzwierciedlają bardzo dobrą jakość predykcji dla klas faktycznie obecnych w danych.

Skuteczność modelu w zadaniu regresji oceniono za pomocą standardowych miar błędu predykcji:

- Średni błąd kwadratowy (MSE) wyniósł 0,0104;
- Pierwiastek średniego błędu kwadratowego (RMSE) osiągnął wartość 0,1018;
- Średni błąd bezwzględny (MAE) wyniósł 0,0194

Niskie wartości miar błędów wskazują na wysoką dokładność prognoz oraz niewielkie odchylenia wartości przewidywanych od rzeczywistych. Otrzymane rezultaty potwierdzają, że zaprojektowany model skutecznie odwzorowuje zależności czasowe w danych ruchu miejskiego i może być wykorzystywany do krótkoterminowego prognozowania natężenia ruchu.

CLASSIFICATION REPORT:					
		precision	recall	f1-score	support
	good	0.99	1.00	0.99	4859910
	serious	0.91	0.84	0.87	290682
	severe	0.00	0.00	0.00	0
	accuracy			0.99	5150592
	macro avg	0.63	0.61	0.62	5150592
	weighted avg	0.99	0.99	0.99	5150592
REGRESSION METRICS:					
MSE = 0.0104					
RMSE = 0.1018					
MAE = 0.0194					

Rysunek 2. Wyniki modelu, źródło: opracowanie własne.

Confusion Matrix				
True label	good	serious	severe	
	99.53% (4,837,082)	0.47% (22,828)	0.00% (0)	
	16.44% (47,790)	83.56% (242,892)	0.00% (0)	
	0.00% (0)	0.00% (0)	0.00% (0)	
		good	serious	severe
		Predicted label		

Rysunek 3. Confusion Matrix, źródło: opracowanie własne.

Rysunki przedstawiają wyniki ewaluacji modelu predykcji natężenia ruchu: znormalizowana macierz pomyłek oraz raport klasyfikacji

5. PODSUMOWANIE

Celem pracy było opracowanie modelu prognozującego zatłoczenie w ruchu miejskim z wykorzystaniem rekurencyjnych sieci neuronowych. Przeprowadzone badania wykazały, że zastosowane podejście umożliwia skuteczne modelowanie zależności czasowych w danych ruchu drogowego oraz generowanie trafnych prognoz krótkoterminowych. Uzyskane wyniki potwierdzają, że metody głębokiego uczenia mogą stanowić wartościowe narzędzie wspierające analizę i interpretację dynamiki ruchu miejskiego

5.1. Dyskusja

Otrzymane wyniki odnoszą się bezpośrednio do postawionego celu badawczego oraz pytania o możliwość wykorzystania rekurencyjnych sieci neuronowych do prognozowania zatłoczenia w ruchu miejskim. Wysoka jakość predykcji wskazuje, że model skutecznie uchwycił zależności czasowe obecne w danych. Należy jednak podkreślić, że skuteczność rozwiązania jest silnie uzależniona od jakości i kompletności danych wejściowych, a brak reprezentacji niektórych klas może wpływać na interpretację wyników. W szerszym kontekście uzyskane rezultaty potwierdzają potencjał metod uczenia głębokiego w zastosowaniach transportowych oraz ich przydatności w systemach inteligentnego zarządzania ruchem.

5.2. Wnioski

Na podstawie przeprowadzonych badań stwierdzono, że rekurencyjne sieci neuronowe mogą być skutecznie wykorzystywane do prognozowania natężenia ruchu miejskiego. Odpowiadając na pytanie badawcze, zastosowany model pozwolił na odwzorowanie zależności czasowych w danych oraz generowanie trafnych prognoz. Postawiony cel pracy został osiągnięty, a uzyskane wyniki potwierdzają zasadność dalszego rozwoju tego typu rozwiązań w ramach koncepcji smart city.

5.3. Kierunki dalszych badań

W przyszłych pracach badawczych możliwe jest rozwinięcie zaproponowanego rozwiązania poprzez zastosowanie większej mocy obliczeniowej, co umożliwiłoby realizację predykcji w trybie ciągłym i zbliżonym do rzeczywistego. Istotnym kierunkiem rozwoju jest także rozszerzenie danych wejściowych o dodatkowe źródła informacji, takie jak dane meteorologiczne, informacje o wypadkach.

Literatura

1. Xiaolei Ma, Zhimin Tao, Yinhai Wang, Haiyang Yu, Yunpeng Wang: Long short-term memory neural network for traffic speed prediction using remote microwave sensor data, 2015
2. Haibo Chen, Susan Grant-Muller: Use of sequential learning for short-term traffic flow forecasting, 2001